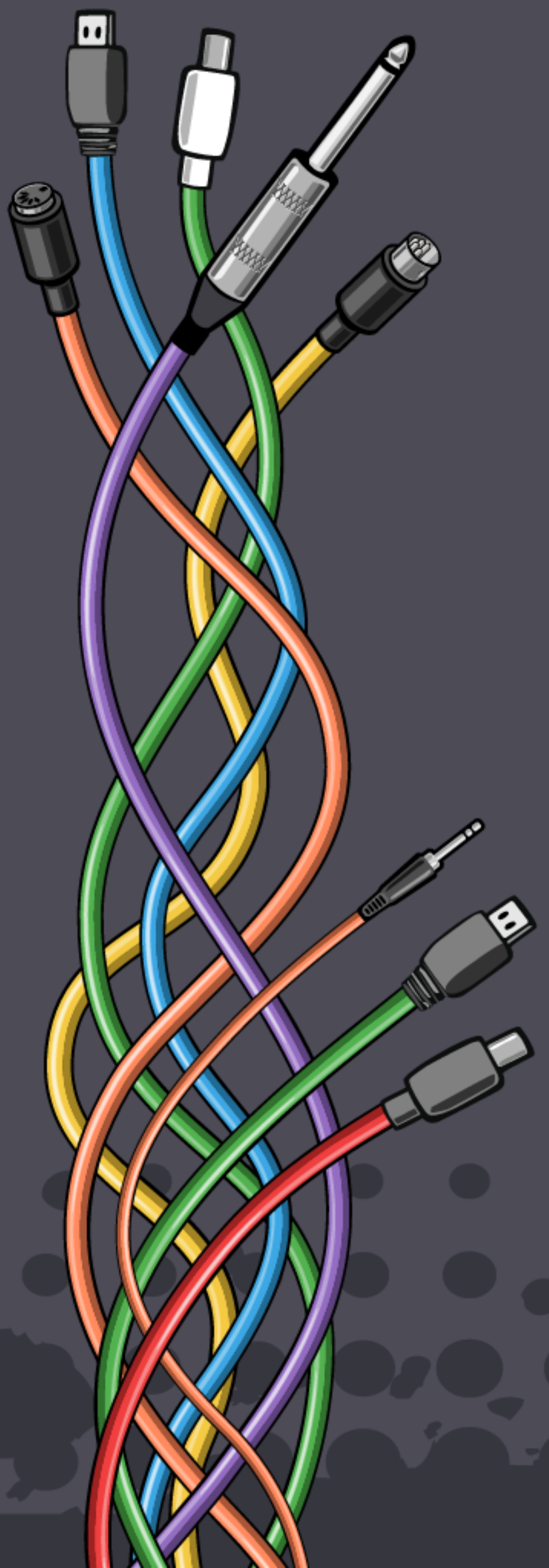


WORKSHOP:

PROGRAMMING MUSIC AND SYNTHESIZERS ON-THE-FLY WITH PHARO

DOMENICO CIPRIANI

ADC25 Bristol

(temporary) AGENDA

1. Foundations [45 minutes]
2. Rhythm and Melodies [75 minutes]

☕ coffee break! (10 minutes)

3. Sound Design with Phausto [75 minutes]
4. Performance [20 minutes]
5. Visual Interface and Control [25 minutes]

GOALS

1. Become friends with live-coding
2. Meet and install Pharo, Coypu and Phausto
3. Create rhythm and melodies with Coypu
4. Create a small set of synthesisers with Phausto and perform with them.
5. Design a basic User Interface with Bloc/CoypuIDE

DISCLAIMER/1

Coypu and Phausto are still in an alpha stage, so we may encounter a few quirks .

Thanks for your patience and sense of adventure!

We're improving together, your feedback makes all the difference. 🚀.

We hope this new syntax and tools make for an inspiring experience.

DISCLAIMER/2

Today we are learning useless stuff,

But that's how useful discoveries often begin.

About me

Domenico Cipriani
a.k.a. Lucretio (the Analogue Cops / Rage Therapy)

- Dj/live performer/live coder/electronic music producer
  - (Tresor, Berghain, Fabric, Rex, Concrete, Faust, LoveParade 2006, Dimensions Festival, Twisted Pepper… Barcelona, Frankfurt, Rome, Hanoi, San Francisco, Detroit, Belgrade, Tokyo, Osaka, Kiev…)
  - More than 150 vinyl releases including records with Blawan, Objekt …
  - Performed at ICLC24 and ICLC25 (+workshop).
- Currently researching in computer music for the Evref team of Inria
- Educator and Artist Partner for Soundwave, Italian distribution of Elektron machines

PART 1 : FOUNDATIONS

What is live coding?

- Live coding is about people interacting with the world, and each other, in real time, via code.
- Live coding is about making software live.
- Live coding is a performance practice that operates as an adventure and exploration, deliberately rejecting fixed definitions, remaining heterogeneous in nature.
- By refusing a fixed definition, live coding also refuses hierarchy, remaining beyond the ownership or control of any institution or established practitioner.

TOPLAP

https://blog.toplap.org

(Temporary|Transnational|Terrestrial|Transdimensional) Organisation
for the
(Promotion|Proliferation|Permanence|Purity) of
Live
(Algorithm|Audio|Art|Artistic)
Programming

informal organization formed in 2004 to bring together the communities formed around live coding environments.

TOPLAP draft manifesto

https://blog.toplap.org

- Give us access to the performer's mind, to the whole human instrument.
- Obscurantism is dangerous. Show us your screens.
- Programs are instruments that can change themselves
- The program is to be transcended - Artificial language is the way.
- Code should be seen as well as heard, underlying algorithms viewed as well as their visual outcome.
- Live coding is not about tools. Algorithms are thoughts. Chainsaws are tools. That's why algorithms are sometimes harder to notice than chainsaws.

What is an algorave?

- An ***Algorave*** is an event where people dance to music generated by algorithms, often performed through live coding
- Born in London in 2011 from the minds of **Alex McLean** and **Nick Collins**, it has since evolved into a worldwide community celebrating the art of coding sound and visuals.
- Code should be projected to the audience.
- Any ***Algorithmic Music*** *i*s welcome when it is

"*wholly or predominantly characterised by the emission of a succession of repetitive conditionals*"

Thinking in many tongues

just to name a few

TidalCycles	Haskell-based language for pattern-based algorithmic music.
ChucK	Strongly timed language for on-the-fly sound synthesis and composition.
Sonic Pi	Accessible Ruby environment for live-coded music and education.
ORCΛ	Esoteric, grid-based environment for composing and sequencing live.
Strudel	Web version of TidalCycles for browser-based live coding.
FoxDot	Python live coding interface for SuperCollider.
Overtone	Clojure environment for synthesis, sampling, and collaborative music.
Mercury	Minimal language for web browser or MasMSP
Extempore	Lisp-like language for real-time programming of sound and interaction.
Hydra	Browser-based system for coding visuals and video synthesis live.

Pharo, Coypu, Phausto

PHARO:
Free, open-source, general-purpose language + cross-platform IDE

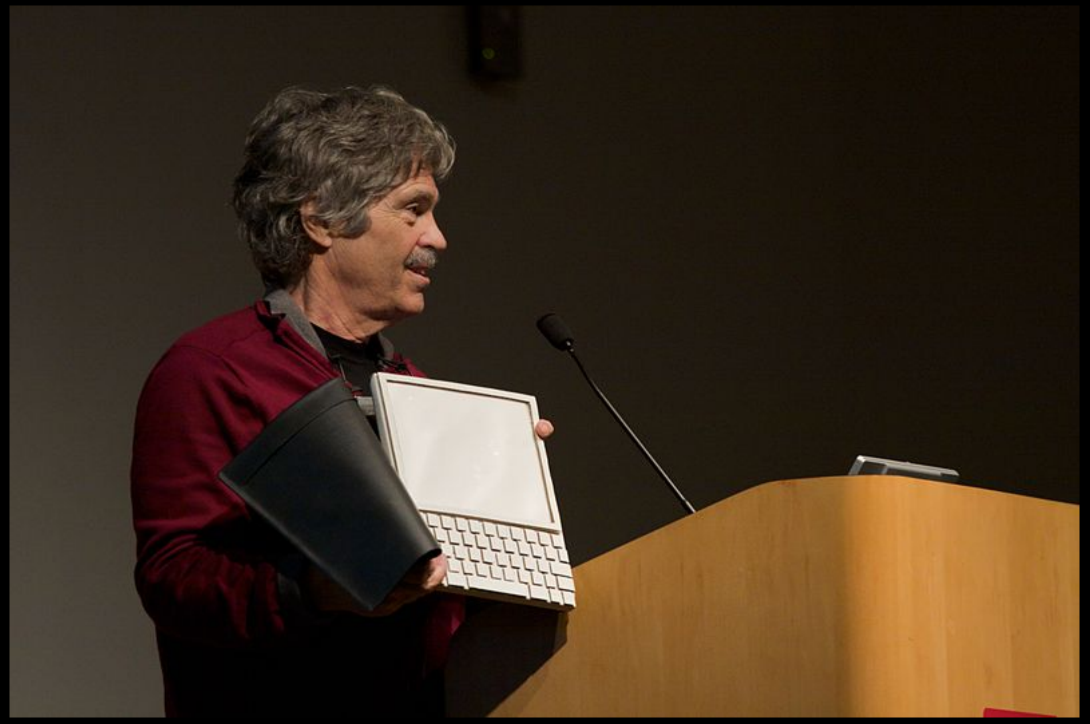
COYPU:
API and Domain Specific Language for programming music on-the-fly

Play synths from

PHAUSTO:
Library and API for DSP programming that enables sound generation within Pharo

Smalltalk

- A pure **Object-Oriented** Programming language developed at the *Learning Research Group* at Xerox PARC in the 1970s by Alan **Kay**, **Dan Ingalls**, **Adele Goldberg**, **Ted Kaeheler**.
- Designed for educational use following principles of *Constructionism*.
- Deeply influenced by ***Simula***, developed by **Ole-Johan Dahl** and **Kristen Nygaard** in the 1960s at the Norwegian Computing Center in Oslo.
- Not just a language, also an IDE users can inspect and modify.
- Programs written in Smalltalk are compiled into byte code and interpreted by a virtual machine.
- Smalltalk has influenced *Objective C*, *Ruby, SuperCollider*.

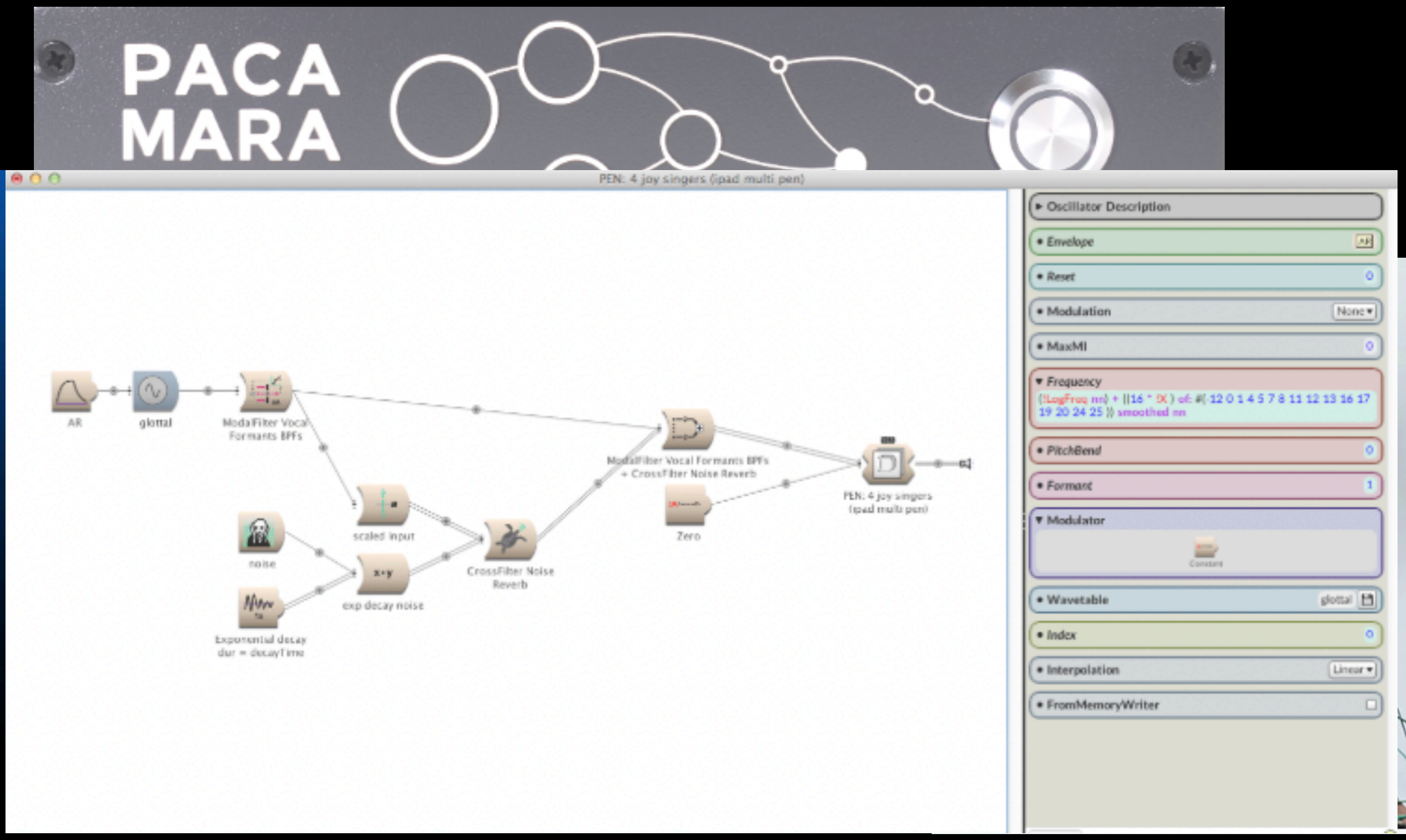


Pharo

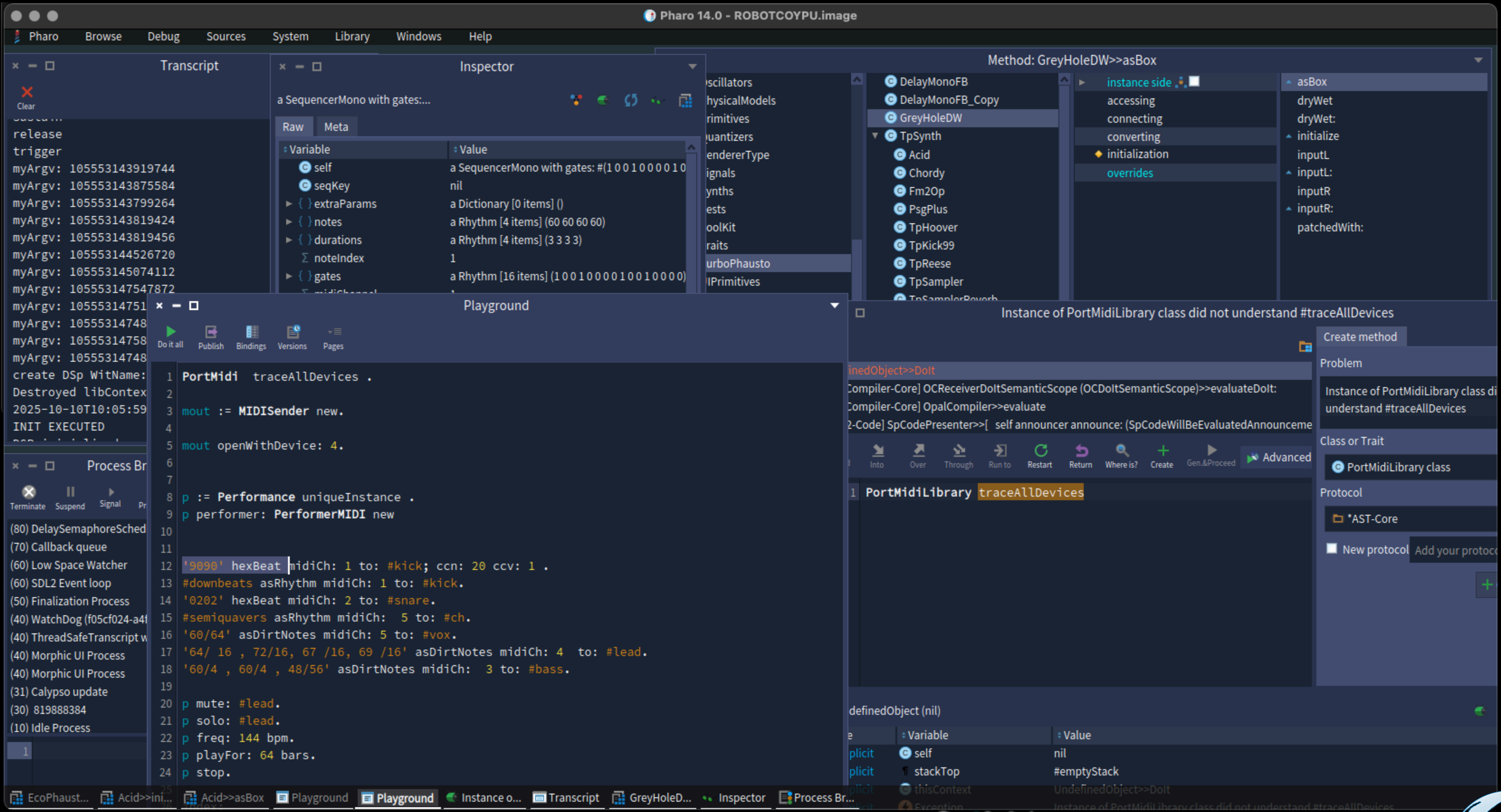
- A pure object-oriented, dynamically typed, and reflective language with a platform independent IDE
- Pharo was born as a fork of *Squeak Smalltalk*, led by **Stéphane Ducasse** and **Marcus Denker** at **INRIA Lille**.
- It is currently developed by INRIA, the Pharo Consortium, and a vibrant worldwide community
- Its syntax fits in a postcard.
- Integrated *Git* support and a framework for SUnit Tests.

Why Pharo?

- Born during the COVID-19 pandemic, **Coypu** grew from experiments in **Kyma**, whose closed-source limits inspired us to create an open-source OSC-based client.
- I chose Pharo for its modern design, extensive online resources, and approachable learning curve, and for its reflective, dynamic, and interactive programming model.



The Pharo IDE



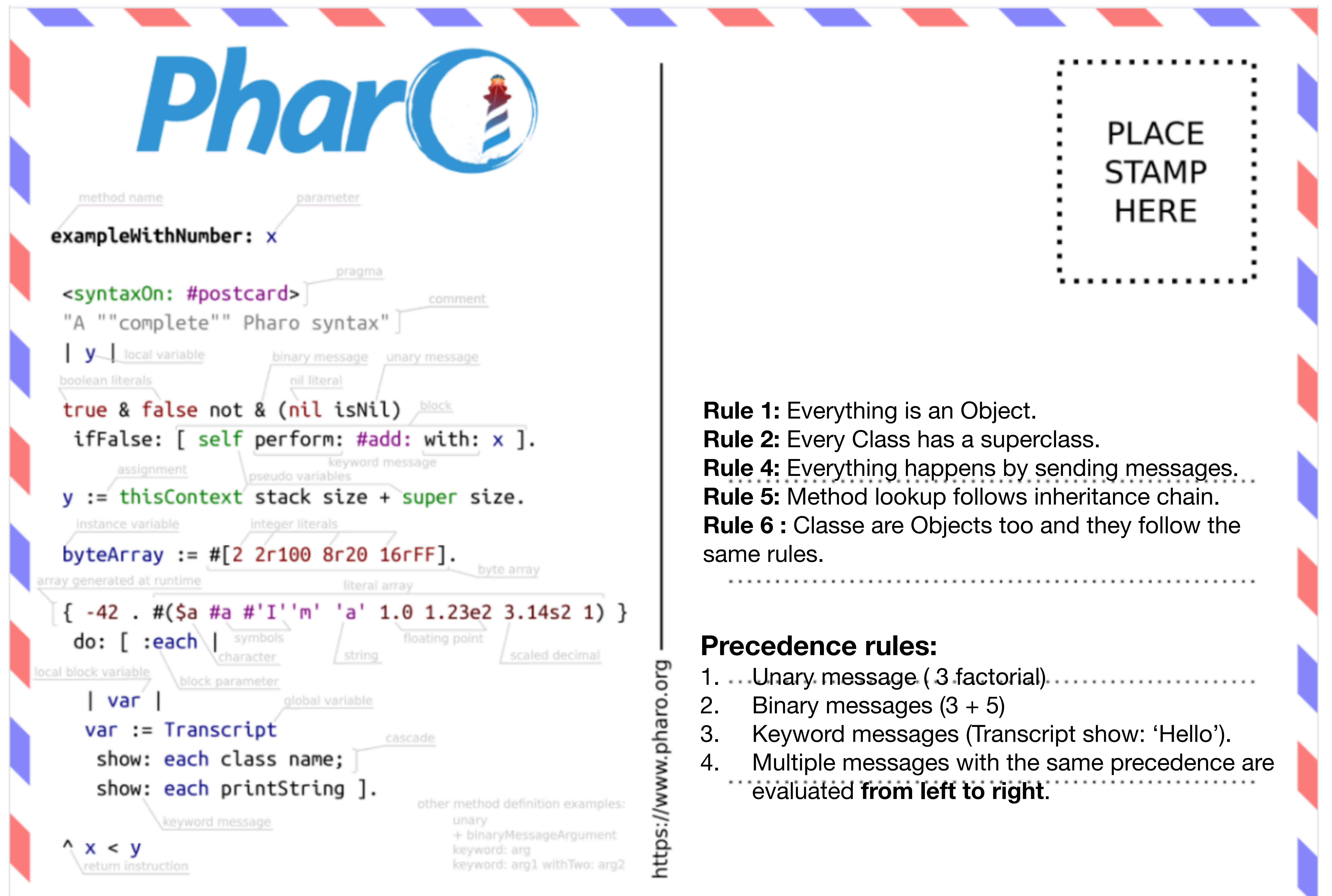
Install Pharo

Download the Pharo Launcher to download a Pharo Image:

https://pharo.org/download

A Pharo image is an object space + Pharo Core Libraries + the virtual machine

Pharo syntax on a postcard



The Pharo IDE / shortcuts

CMD/CTRL + OP	Open Playground
CMD/CTRL + D	Evaluate (Do)
CMD/CTRL + P	Print Object
CMD/CTRL + I	Inspect Object
CMD/CTRL + B	Browse Object
CMD/CTRL + M	List Implementors

Installing Coypu and Phausto

Go to the *Coypu* GitHub repositories:

https://github.com/lucretiomsp/Coypu

Copy the *Metacello* script into your *Playground*.

```
Metacello new
    baseline: 'Coypu';
    repository: 'github://lucretiomsp/coypu:master';
    load
```

Select all the text and evaluate *(CMD/CTRL + D)*
***[*Coypu* already comes together with *Phausto*]**

PART 2 :
RHYTHM AND MELODIES

Coypu's anatomy

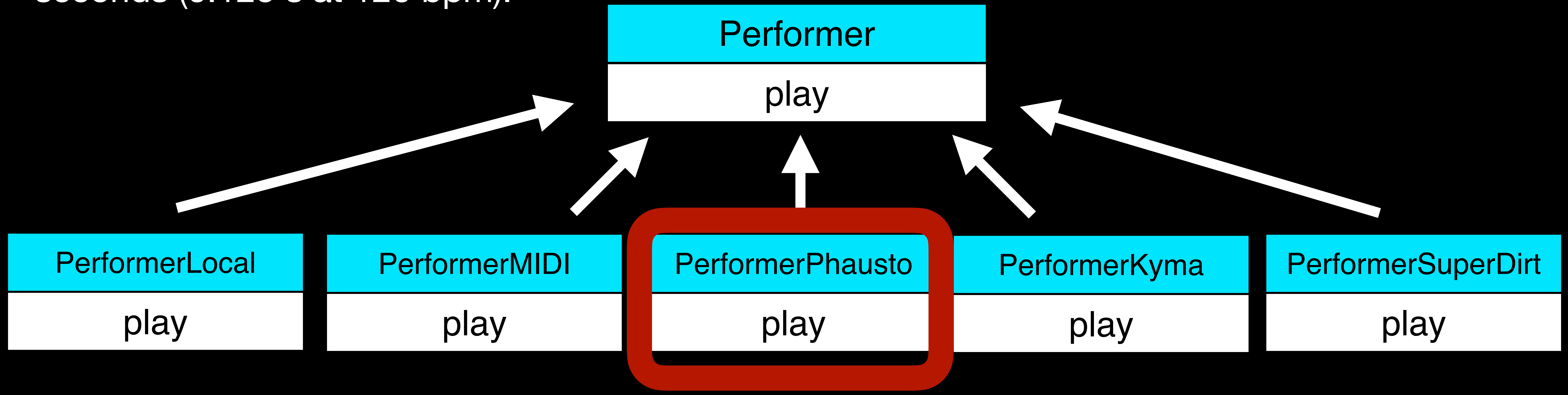
Coypu

Domain Specific Dialect

- Play sounds from an audio server or MIDI instrument
- Create, transform and manipulate Rhythms
- Create, transform and manipulate melodies, chords and arpeggios
- Render visuals from a graphics engine or visual synthesizer.
- Create custom OSC controllers

Coypu's anatomy

- The **Performance** class is a *Singleton*.
- A **Performer** must be assigned to the Performance; the Performer selects the audio backend.
- The *play* method starts the Performance, and increments the transportStep every `freq` seconds (0.125 s at 120 bpm).



Coypu's anatomy

transportStep increments of 1 every **Performance** uniqueInstance freq (seconds)

PERFORMANCE

SEQUENCER key: #kick

SEQUENCER key: #bass

SEQUENCER key: #snare

SEQUENCER key: #lead

SEQUENCER key: #perc

SEQUENCER

- **gates**
- **notes**
- **durations**
- **dirtMessage**

Principles

Iconicity

Written code should resemble what we hear

```
16 upbeats
```

Economy

The less we type, the better

```
#(60 63 67) + 16
```

Polysemy

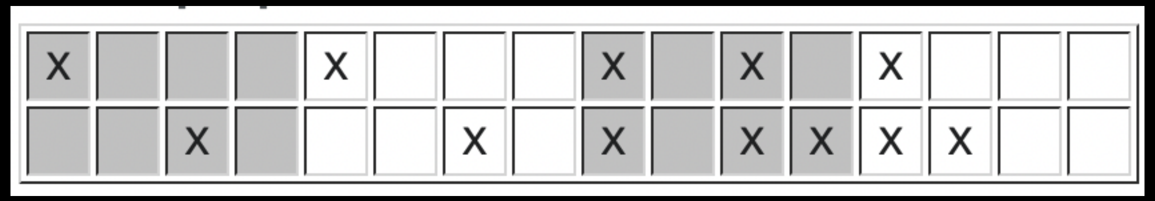
Many ways to do the same thing

```
16 downbeats.
#downbeats asRhythm.
'60 , ~ , ~ , ~ , 60 , ~ , ~ , ~ , 60 , ~ , ~ , ~ , 60 , ~ , ~ , ~' asDirtNotes.
#(1 0 0 0 1 0 0 0 1 0 0 0 1 0 0 0 1 0 0 0) asSeq.
```

Creating rhythms

- Sequencers are inspired by traditional hardware sequencers, where triggers (~noteOn) events are notated in **Time Unit Box System** (TUBS).

TUBS



Traditional

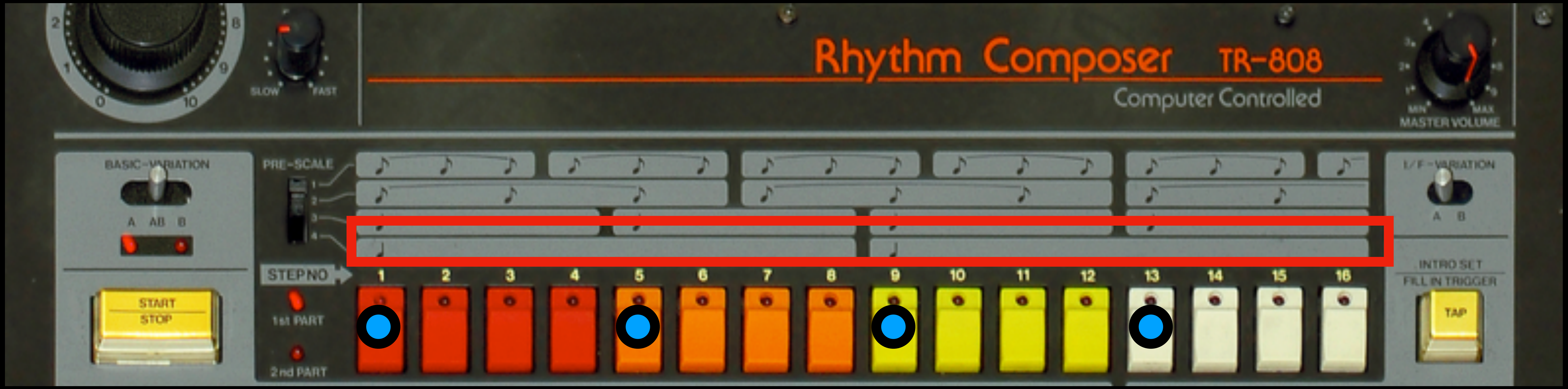
Coypu

#(1 0 0 0 1 0 0 0 1 0 1 0 1 0 0 0) asSeq

- Performance can be thought as a multitrack player for Sequencers.
- Sequencer are filled with parameters.
- **A Performer must be assigned to the Performance.**

```
p := Performance uniqueInstance.  “assign the unique instance of the Performer class
a variable”
p performer: PerformerLocal new. “assign a local OSC performer to the Performance”
p freq: 136 bpm. "change the performances speed"
#( 1 0 0 0 1 1 0 0 1 0)  asSeq notes: ’32 37 38’ to: #bass. “create a Sequencer and
assign it to the performance”
p playFor: 64 bars. “play the performance for 64 bars".
p stop. "stop the performance"
```

Creating rhythms/2



A rhythm can be represented as an array of 0s and 1s, where each 1 represents a trigger.

PHARO	#(1 0 0 0 1 0 0 0 1 0 0 0 1 0 0 0)
COYPU	#downbeats asRhythm
COYPU	16 downbeats
COYPU	‘8888’ hexBeat

Ponle la gasolina!

(We will talk about Phausto after the break, now let's give it some gas! 🏁)

```
TurboPhausto start.
tp := TurboPhausto new.
tp bpm: 167.

#junglekick' asRhythm to: #Kick.
16 cumbiaClave to: #Marimba.a
#(5 16) euclidean to: #Conga.

tp play.
```

```
EcoPhausto start.
ep := EcoPhausto new.
ep bpm: 137.

'9090' hexBeat to: #Kick.
16 quavers to: #Ch.
16 randomTrigs to: #Bongo.

tp play.
```

Turbo/EcoPhausto

- TurboPhausto is set of synthesisers and effects, *made with Phausto*, crafted for programming music on-the-fly with Coypu.
- Inspired by the SuperDirt audio engine for SuperCollider, it turns Pharo into a powerful environment for live-coded music and algoraves.
- Packed with 50 MB of algorave-ready royalty free audio samples, made by *Lucretio*, *The Analogue Cops* and the legendary dutch electro producer *Legowelt*.

Creating rhythms/3

The name of a rhythm
(*the message*)

16 quavers

Integer number of steps on the Sequencer
(*the receiver*)

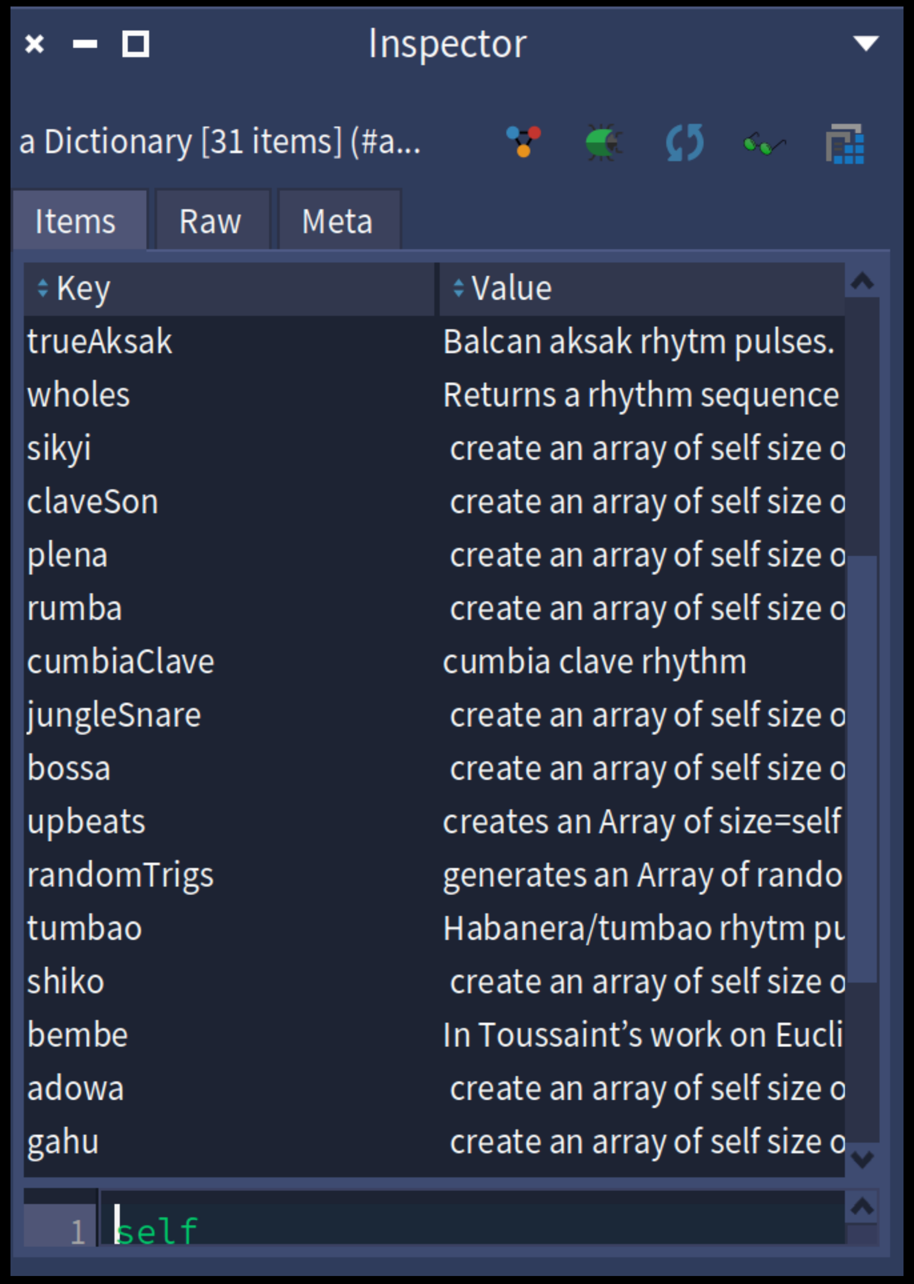
=

(*the message*)

#quavers asRhythm

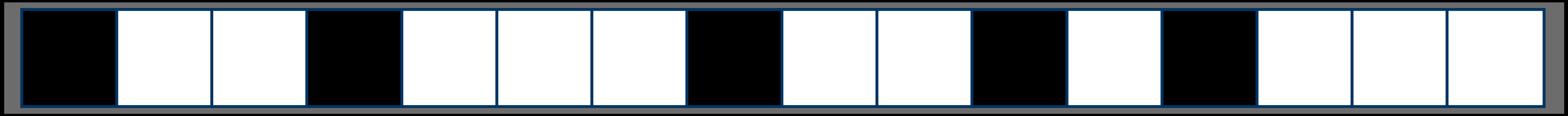
The name of a rhythm preceded by # - becomes a symbol -
(*the receiver*)

Rhythm list inspect

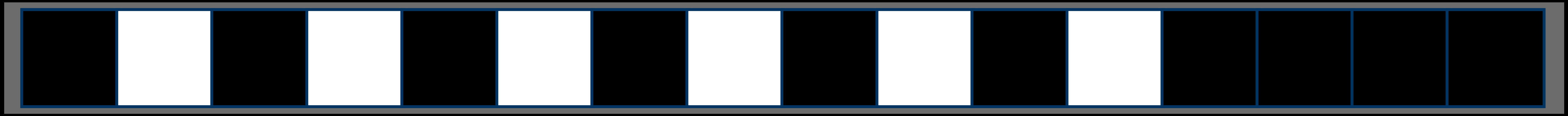


Exceptions!
4 breves.
8 semibreves.
(Here, the integer receiver specifies the number of notes with that duration.)

Creating rhythms/4



- #(1 0 0 1 0 0 0 1 0 0 1 0 1 0 0 0) asSeq
- #rumba asRhythm
- ‘9128’ hexBeat



- #(1 0 1 0 1 0 1 0 1 0 1 0 1 1 1) asSeq
- 12 quavers, 4 semiquavers
- ‘AAAF’ hexBeat

Hexbeats

- 1 Bar = 16 Steps (1/16th quantisation).
- If every step corresponds to a Bit of Information, in a Bar we find 16 Bits, (2 Bytes).
- In every Bar there are 4 Beats, so in each Beat there are 4 Bits, i.e. 1 Nibble.
- Every Hexadecimal symbol represents a Nibble (4 steps)

0	0	0	0	0	
1	0	0	0	1	
2	0	0	1	0	Upbeats
3	0	0	1	1	
4	0	1	0	0	
5	0	1	0	1	
6	0	1	1	0	
7	0	1	1	1	
8	1	0	0	0	Downbeats
9	1	0	0	1	
A	1	0	1	0	Quavers
B	1	0	1	1	
C	1	1	0	0	
D	1	1	0	1	
E	1	1	1	0	
F	1	1	1	1	Semiquavers

Euclidean rhythms

- Godfried Toussaint showed that many traditional rhythms follow patterns generated by Euclid's algorithm.
- Euclidean rhythms distribute beats as evenly as possible across a set number of steps.
- Technically, they are created by applying the Bjorklund algorithm, which spaces ***k*** onsets evenly over ***n*** pulses.

#(5 16) euclidean to: #cowbell

Creating rhythms/5

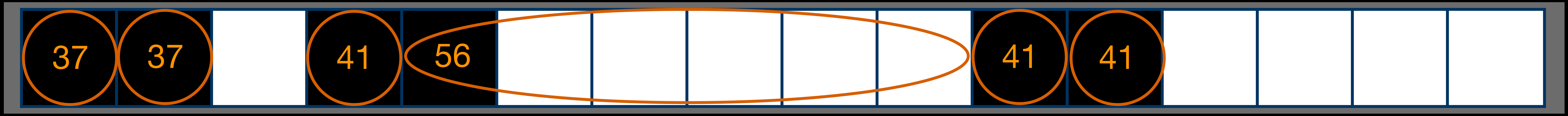
By default, every note in a sequencer is triggered using MIDI note number 60 (Middle C, or C4). The duration of each note is calculated as *sequencer size / number of trigs*, with a gate time of 0.8.

We can define arrays of note durations and/or gate times. These arrays don't need to have the same length as the number of trigs, which allows us to create polymetric sequencers—sequencers where each parameter can have its own rhythmic cycle.

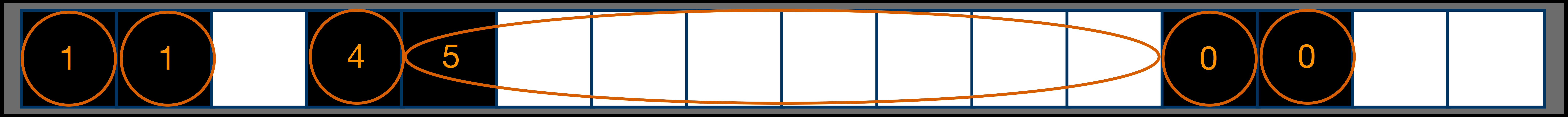
For example:
16 semiquavers notes: '38, 41 , 43' ; durations: '0.1 , 0.3' ; gateTime: '0.6, 0.5 , 0.4 , 0.1'

The ‘musky’ notation

A Tidal Cycles ‘string notation’ to create Sequencers with different notes or different samples



- ’37 * 2 , ~ , 41 , 56 / 8 , 41* 2 , ~ * 4 ’ asDirtNotes



- ’1 * 2 , ~ , 4 , 5 / 8 , 0* 2 , ~ * 2 ’ asDirtIndex

Random rhythms and melodies

```
16 randomRhythm.
32 randomTrigs.
64 randomTrigWithProbability: 70.
24 plena randomNotesIn: Scale istrian + 46.
(32 semiquavers randomNotesFrom: #(48 60 63 56)).
(64 semiquavers randomWalksOn: Scale sakura  octaves: 3  root: 48).

gen := MarkovJazzMelodyGenerator new.
melody := gen generateMelodyFromScaleDegrees: Scale bebop root: 60 startDegree: 0 length: 32.
```

Scales

- Sending a scale name to the **Scale** class returns an array of intervals for that scale.
- You can add a root note using **+** to shift all intervals.
- Or get a full array of notes across octaves by sending:

  Scale root: <rootNote> octave: <n>

  This gives all the notes starting from the root across n octaves.

```
• Scale sakura ^ #(0 1 5 7 8).
• Scale sakura + 60 ^ #(60 61 65 67 68).
• Scale sakura root: 36 octave: 2 ^ # (36 37 41 43 44 48 49 53 55 56 60 61 65 67 68)
```

Sequencer operations

Combine Sequencers with ,

```
#junglekick' asRhythm , #downbeats asRythm , 'AAAF' hexBeat
```

Repeat Sequencers with *

```
'AAAF' hexBeat * 3
```

Transpose Sequencers with +

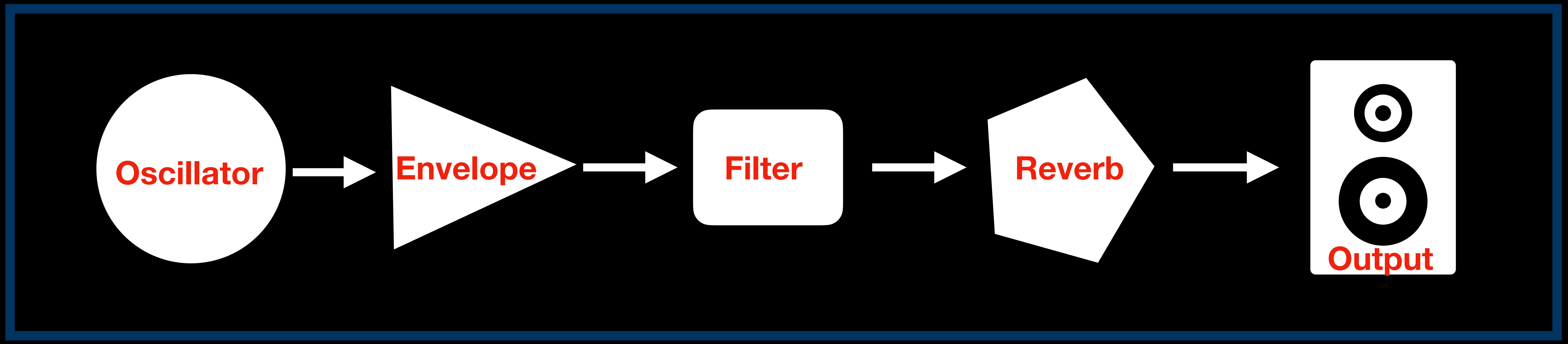
```
#semiquavers hexBeat + 7
```

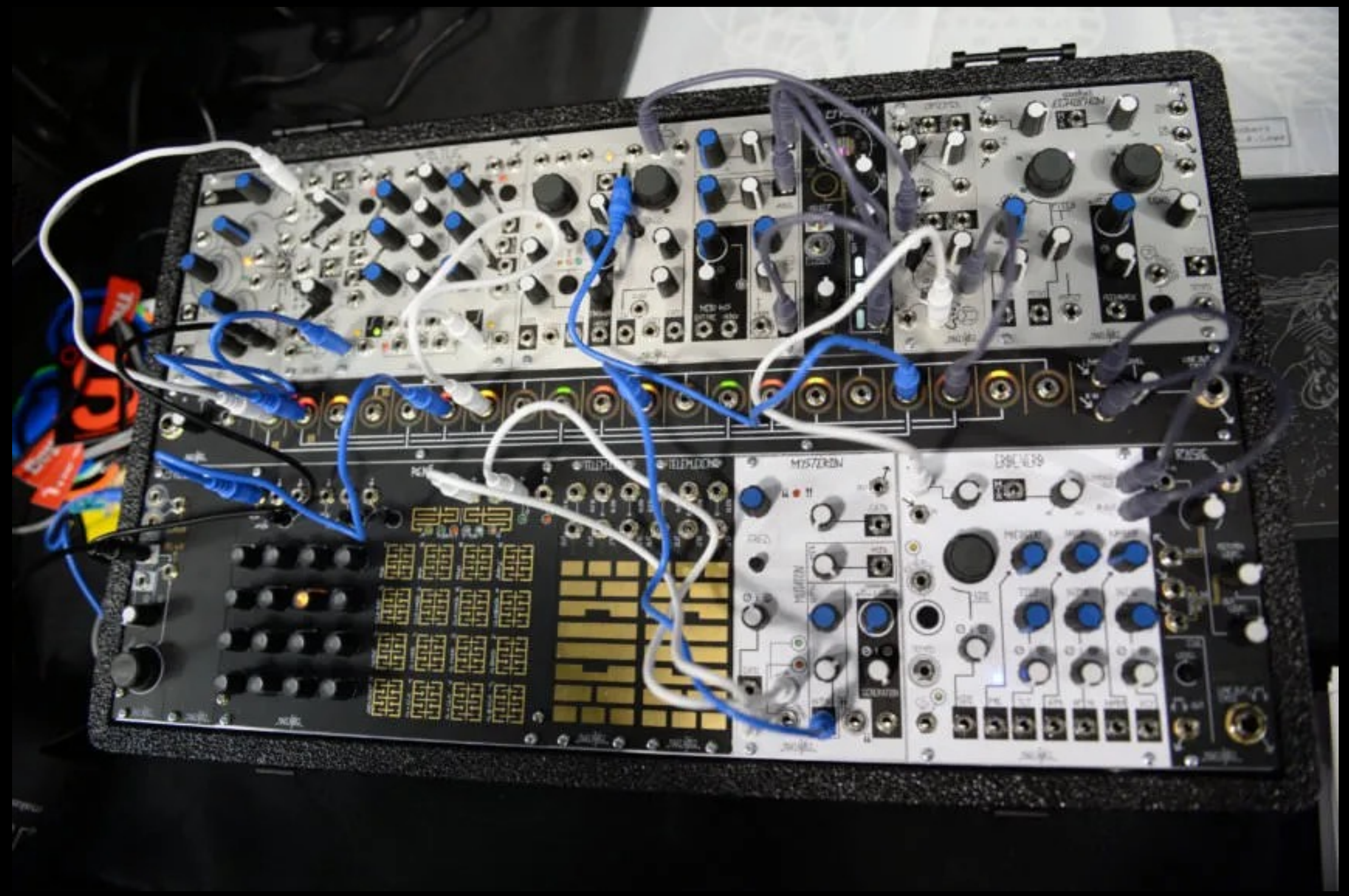
PART 3 : SOUND DESIGN WITH PHAUSTO

Domenico Cipriani - ADC25

Start your engine

- Phausto is a library and API for sound generation and DSP programming within the Pharo IDE, powered by an embedded Faust compiler.
- Phausto brings a modular-synth-inspired approach to designing synths and effects.
- Unit Generators are connected by setting their members value or using the ChucK operator => .





Let's build a synth

```
oscillator := PulseOsc new label: 'MySynth'.
env := ADSREnv new label: ‘MySynth'.
filter := Korg35LPF new label: 'MySynth'.
effect := GreyHoleDW new label: 'MySynth'.

synth := oscillator => env => filter => effect.
dsp := synth stereo asDsp.

dsp traceAllParams.
dsp init.
dsp start.
dsp playNote: 60  prefix: 'MySynth'  dur: 1.
```

Drive Phausto with Coypu

```
p := Performance uniqueInstance .
p forDsp: dsp.

'38, 56, 65, ~ , 42 , 76 , 81 , 45' asDirtNotes to: #MySynth.

p freq: 129 bpm.
p playFor: 16 bars.
```

Sum of synths

```
dsp := (Kick new + Clap new + CombString new + Hat new + PopFilterDrum new + SawTrombone new + DubDub new) stereo asDsp.

dsp init.
dsp start.

p := Performance uniqueInstance .
p forDsp: dsp.

'8080' hexBeat to: #Kick.
'38, 56, 65, ~ , 42 , 76 , 81 , 45' asDirtNotes to: #DubDub.
#rumba asRhythm to: #Hat.
32 randomTrigs to: #Clap.
'60/32 , 63/16 ' asDirtNotes to: #SawTrombone.
#( 7 16 euclidean) to: #PopFIlterDrum.

p playFor: 16 bars.
```

MasterLu

Our First Synthesiser (8/15)

Do it all Publish Bindings Versions Pages

```
"lets create our first subtractive synthesiser"

"first we need at least one oscillator"
oscillator := PulseOsc new.
"then we need an envelope, in this case a linear Attack-Decay-Sustain-Release envelope"
envelope := ADSREnv new.
"also we create a filter, in this case a Moog Vcf lowpass filter"
filter := MoogVcf new.

"to construct our synthesyser we chuck the Oscillator into the Envelope (hence the Oscillator is multiplied by the evelope - and we chuck this composite Unit Generator into the filter, this means that the composite Unit Generator is the input of the filter".

synth := oscillator => envelope => filter.
"binary operators have left-to-right precedence in Pharo, so we just place the Unit Generators in chain"

"now we create a steereo DSP"
dsp := synth stereo asDsp.
"we initialize the DSP"
dsp init.
"we start it"
dsp start.

"to hear the sound we need to trigger the envelope, so we must open the UI and press the button"
dsp displayUI .

"now have fun tweaking the sliders, try to understand what happens to the sound".

"and then we stop the DSP before jumping to the next lesson"
dsp stop.
MasterLu next.
```

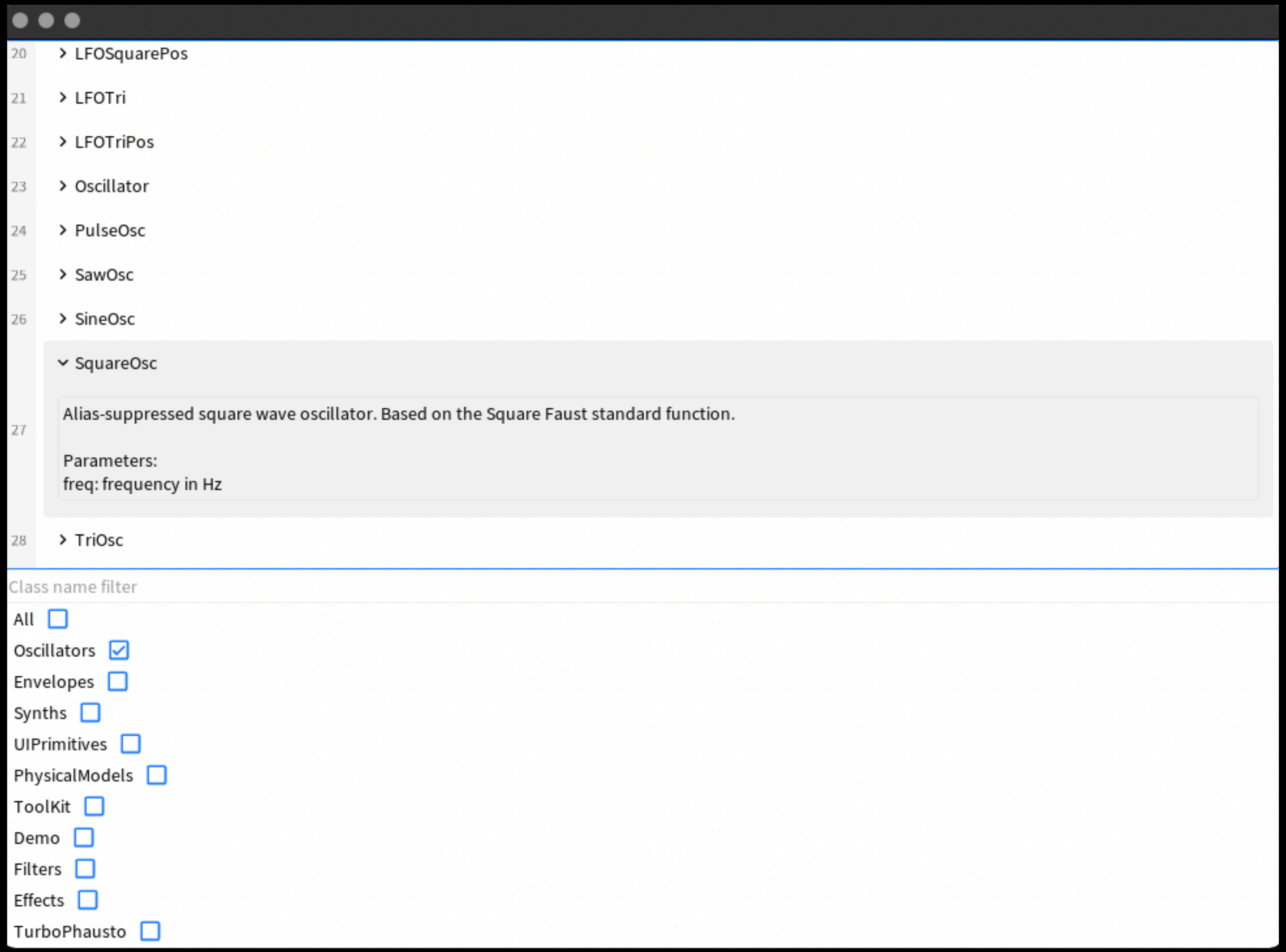
Line: 29:15

```
Metacello new
    baseline: 'MasterLu';
    repository: 'github://lucretiomsp/MasterLu:main';
    load
```

```
MasterLu go.
MasterLu next.
```

***MasterLu** is your electronic music teacher. 15 lesson everything you need to know*

ClassReference




```
PhaustoClassReference new openInSpace.
```

Build your CustomPhausto

Exercise!

- *Study the **EcoPhausto** class and design a subclass called **CustomPhausto**.*
- *Your design should include multiple sound sources, at least two **TpSamplers** and two **Synths,**drawn from Phausto's existing set or custom ones you define.*
- *When your **CustomPhausto** is complete, we'll be ready to perform with it and **Coypu**.*

PART 4 :
PERFORMANCE

Tuning Phausto for Coypu

```
tablas := TpSampler new label: 'tablas' ; pathToFolder: '/samples/tabla'.
viola := TpSampler new label: 'viola' ; pathToFile: '/samples/strings/viola.wav'.
pulsar := (PulseOsc new label: 'pulsar') => ADSR new label: 'pulsar';
synths :=  (table + viola + pulsar) stereo asDsp.
```

⚠️ ***Labels should begin with minor case letters!***

Nibbling things up

```
p := Performance uniqueInstance .
"Any Performer can be used"

p freq: 129 bpm.
p playFor: 128 bars.

p mute: #kick.
p mute: #(#clap #bass).

p solo: #bongo.
p unsolo: #bongo.

p backup.

p muteAll.
'AAAAAAAF' hexBeat to: #snare.
p restore
```

Nibbling things up/2

```
EcoPhausto start.
ep := EcoPhausto new.

ep playFor: 128 bars.

downbeats #asRhythm to: #kick.
rumba #asRhythm to: #kick.
#kick swapWith: #snare.
ep solo: #bongo.

ep solo: #bongo.
ep unsolo: #bongo.

ep backup.

#(5 16 euclidean) to: #bongo.
(#(5 16 euclidean) offset: 2)to: #conga. “Also try offsetLeft or offsetRight!”

ep restore.
```

Extemporaneous Music

- We invite you to perform without synchronisation. Follow the sounds coming next to you. Let collective time arise from listening.
- **Extemporaneous Music** explores sound as a spontaneous events, echoing the ideas of sonic pointillism where each sound exists in its own space and time.
- A performance created *ex tempore*, emerges from real-time interaction between performers and machines.
- This approach proposes an alternative musical form: collective, algorithmic, improvised, and continuously unfolding in the moment and alive in the moment

PART 5 : VISUAL INTERFACE

Bloc

- **Bloc** is a powerful low-level graphical framework for Pharo, created by **Alain Plantec** and enhanced by the *Feenk* team for *GToolkit*.
- **Bloc** is expected to become the primary graphical framework for Pharo, gradually replacing the well-established but ageing Morphic framework.
- **Bloc** is under active development, so features and behavior may change

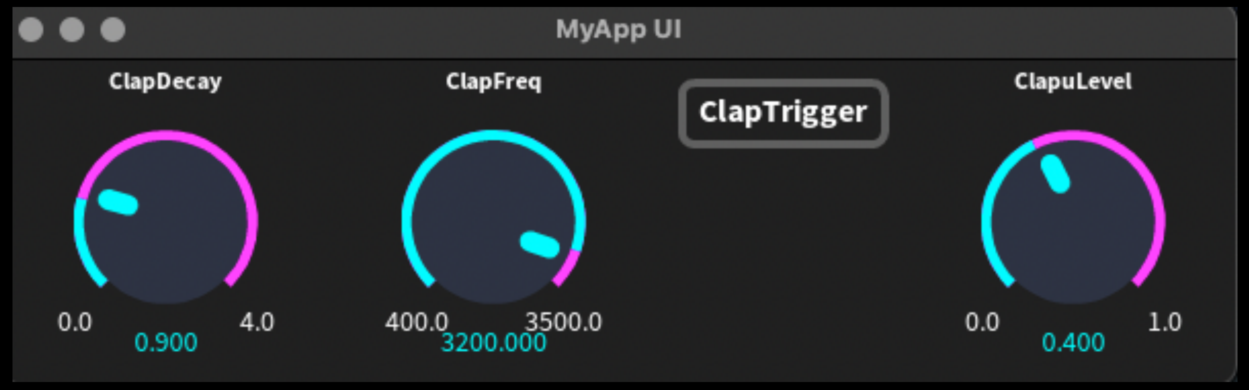
CoypuIDE

- CoypuIDE is a set of widgets for the Coypu and Phausto music and DSP libraries, implemented in Bloc and Toplo.
- It comes together with the MasterLu package or can be installed separately with a script:

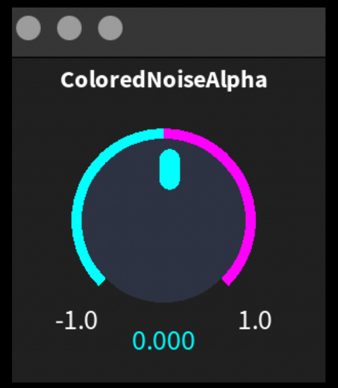
```
Metacello new
    baseline: 'CoypuIDE';
    repository: ‘github://pharo-graphics/COypuIDE:master';
    load
```

CoypuIDE/2

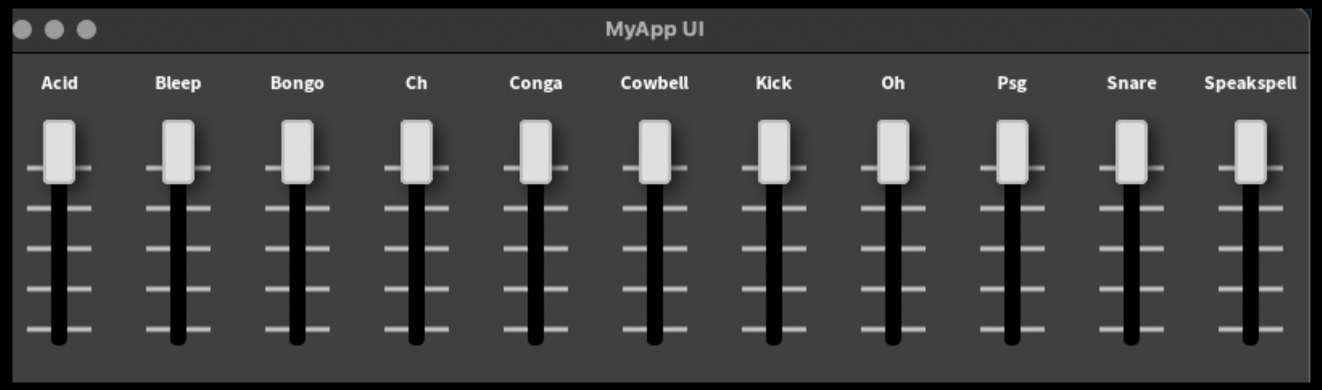
```
dsp := Clap new stereo asDsp.
dsp init; start.
"Display the full UI"
dsp displayUI.
```




```
dsp := ColoredNoise new stereo asDsp.
dsp init; start.
dsp traceAllParams.
widget := dsp widgetForParameter: 'ColoredNoiseAlpha'.
Widget openInSpace.
```

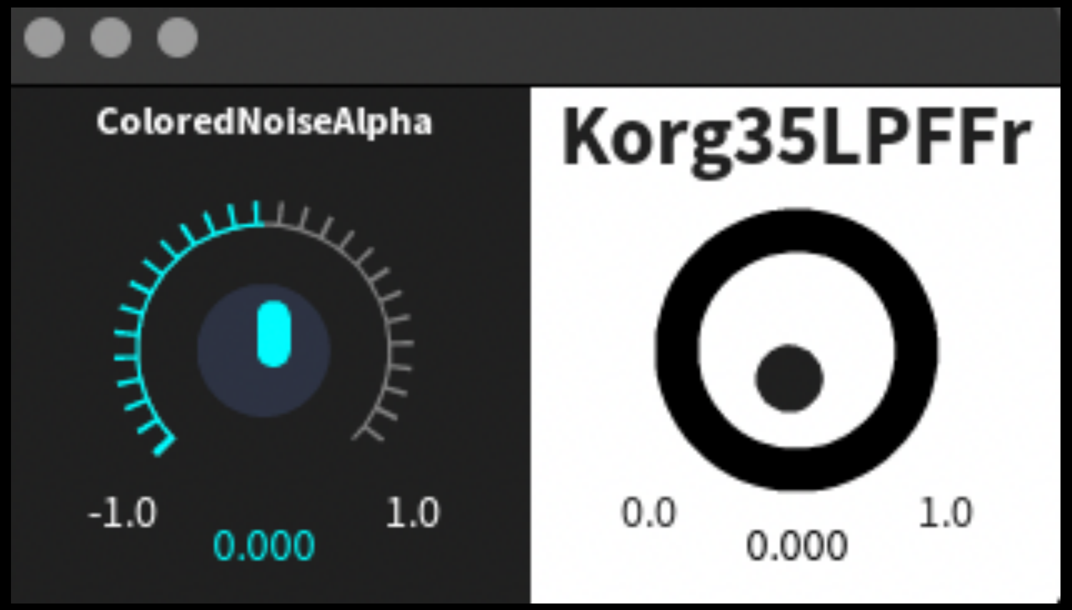



```
EcoPhausto start.
ep := EcoPhausto new.
ep displayUIForLevels.
```



ICContainer

```
dsp := (ColoredNoise new => Korg35LPF new) stereo asDsp.
dsp init; start.
dsp traceAllParams.
knob1 := dsp widgetForParameter: ‘ColoredNoiseAlpha’ type: #ICDialKnob.
knob2 := dsp widgetForParameter: ‘Korg35LPFFr’ type: #ICSimpleKnob.
container := ICContainer new.
container addChild: knob1; addChild: knob2.
container show.
```



Join the community

Pharonauts on Discord